

2026

Software Architecture Specifications

Uni Textbook Marketplace

For **Project Day**

Presented by:

NexusDev

Software Architecture Specifications (SAS)

Project:	Uni Textbook Marketplace
Team:	NexusDev
Client:	Agile Bridge
University:	University of Pretoria - COS 301 Software Engineering 2026
Document version:	3.0 - Demo 4
Last updated:	29 September 2026

The Team

Team Member	Student number
1. Tiego Mokwena*	22496336
2. Gift Mohuba	23545527
3. Neo Bosoga	23591732
4. Josh Kretschmer	22883925

Contents

1 Introduction **6**

2 Architectural Requirements **7**
2.1 Architectural Patterns . . . 7
2.2 Design Patterns . . . 9
2.3 Constraints . . . 9
2.4 Architectural Diagram . . . 10
2.5 Mapping Quality Requirements to Architectural Decisions . . . 11

3 Technology Requirements **13**

4 API Contracts **16**
4.1 Carried Over from Demo 1 (confirmed still in use) . . . 16
4.2 Auth API Contracts . . . 16
4.2.1 POST /auth/register . . . 16
4.2.2 POST /auth/verify-otp . . . 16
4.2.3 POST /auth/login . . . 17
4.2.4 POST /auth/set-password . . . 17
4.2.5 GET /auth/me . . . 18
4.2.6 POST /auth/logout . . . 18
4.2.7 GET /auth/universities . . . 18
4.2.8 POST /auth/refresh . . . 18
4.3 Listings API Contracts . . . 18
4.3.1 POST /listings . . . 18
4.3.2 GET /listings . . . 19
4.3.3 GET /listings/mine . . . 19
4.3.4 GET /listings/:id . . . 19
4.4 Admin API Contracts . . . 19
4.4.1 GET /listings/admin/all . . . 19
4.4.2 PATCH /admin/listings/:id/approve . . . 20
4.4.3 PATCH /admin/listings/:id/reject . . . 20
4.5 Modules API Contract . . . 20
4.5.1 POST /modules . . . 20
4.5.2 GET /modules/faculties . . . 20
4.6 New for Demo 2 . . . 20
4.7 Listing API Contract . . . 20
4.7.1 PATCH /listings/editlist . . . 20
4.8 Module API Contract . . . 21

4.8.1 GET /modules (extended filters) . . . 21
4.9 Saved Searches API Contract . . . 21
4.9.1 POST /saved-searches/ . . . 21
4.9.2 GET /saved-searches/me . . . 22
4.9.3 GET /saved-searches/id . . . 22
4.9.4 DELETE /saved-searches/id . . . 22
4.9.5 GET /saved-searches/me/email . . . 22
4.9.6 GET /saved-searches/me/role . . . 23
4.10 Wishlist API Contract . . . 23
4.10.1 POST /wishlist/:listingId . . . 23
4.10.2 DELETE /wishlist/:listingId . . . 23
4.10.3 GET /wishlist/mine . . . 23
4.11 Admin API Contract . . . 23
4.11.1 GET /admin/audit-log . . . 24
4.11.2 GET /admin/emails . . . 24
4.12 Image API Contract . . . 24
4.12.1 POST /images/upload . . . 24
4.12.2 GET /images/list . . . 24
4.12.3 DELETE /images/delete . . . 24
4.13 Conversations API Contract . . . 25
4.13.1 POST /conversations . . . 25
4.13.2 POST /conversations/mine . . . 25
4.13.3 POST /conversations/:id/messages . . . 25
4.13.4 GET /conversations/:id/messages . . . 25
4.14 Demo 3 API Contracts: . . . 26
4.14.1 POST /reports . . . 26
4.14.2 GET /admin/reports . . . 26
4.15 Admin Moderation API Contract . . . 26
4.15.1 PATCH /admin/reports//dismiss . . . 26
4.15.2 PATCH /admin//ban . . . 26
4.16 Notifications API Contract . . . 27
4.16.1 GET /notifications/mine . . . 27
4.16.2 PATCH /notifications/:id/read . . . 27
4.16.3 PATCH /notifications/read-all . . . 27
4.16.4 DELETE /notifications/:id/delete . . . 27
4.17 Listing Status API Contract . . . 27
4.17.1 PATCH /listings/:id/status . . . 28
4.18 Demo 4 API Contracts . . . 28
4.18.1 POST /vision/extract-text . . . 28

4.18.2 POST /auction . . . 29
4.18.3 GET /auction . . . 30
4.18.4 GET /auction/:id . . . 31
4.18.5 GET /auction/listing/:listingId . . . 32
4.18.6 POST /auction/:id/bid . . . 32
4.18.7 GET /auction/:id/bids . . . 33
4.18.8 Auction closing (background job) . . . 34
4.18.9 POST /bundles/optimize . . . 34
4.18.10 POST /cases . . . 37
4.18.11 GET /cases/mine . . . 38
4.18.12 GET /cases/admin . . . 38
4.18.13 GET /cases/admin/pending . . . 38
4.18.14 GET /cases/:id . . . 39
4.18.15 PATCH /cases/:id/review . . . 39

5 Non-Functional Requirements (NFR) Testing 40
5.1 Testing Overview . . . 40
5.2 NFR Traceability Matrix . . . 40
5.3 Test Results Summary . . . 43
5.4 Performance Results . . . 43
5.4.1 NFR-01: Listing Search Latency . . . 43
5.4.2 NFR-02: Notification Delivery Latency . . . 44
5.5 Security Results . . . 45
5.6 Reliability and Availability Results . . . 46
5.6.1 NFR-12: Core Features Independent of the Messaging Service . . . 46
5.6.2 NFR-13: Warm Replica . . . 47
5.7 Maintainability Results . . . 47
5.8 Accessibility Results . . . 48
5.9 Auditability Results . . . 49
5.10 Testing Tools Used . . . 50
5.11 Known Issues and Follow-Up Actions . . . 50

6 Deployment 52
6.1 Overview . . . 52
6.2 Live URLs . . . 53
6.3 Deployment Diagram . . . 53
6.4 CI/CD Pipeline Diagram . . . 54
6.5 Deployment Requirements Status . . . 55
6.6 Custom Domain and DNS . . . 55
6.7 Rollback Strategy . . . 56

6.8 Issues Encountered and the Decisions That Came Out of Them 56

1. Introduction

The Uni Textbook Marketplace is a web app that lets verified university students buy, sell, and swap second-hand textbooks in a way that's actually organised around modules, instead of just being a generic classifieds board. The SRS explains what the system needs to do. This document is about how we actually built it to do that: the architectural and design patterns we went with, the tech stack, our API contracts, and how everything gets deployed.

Roughly speaking, the backend is a NestJS modular monolith, the frontend is Next.js, and Postgres (hosted through Azure) handles the data. Messaging is kept separate and runs through Firebase Firestore instead. Redis, running as its own container, backs the job queue that opens and closes real-time auctions on time. Azure AI Vision does the text recognition behind AI-assisted listing creation. Everything runs on Azure Container Apps, split into a production and a staging environment, and GitHub Actions handles both the quality checks and the actual deployment. The frontend is also reachable on a custom domain, https://www.unitextbookmarketplace.co.za.

2. Architectural Requirements

2.1 Architectural Patterns

Modular Monolith (Core Backend)

The backend is a NestJS modular monolith. Rather than splitting everything into separate microservices right from the start, all the core domain logic (Auth, Listings, Moderation, Reports and Bans, Modules, Saved Search, Wishlist, Notifications, Audit Log, Auction, Vision, Bundle) lives in one NestJS app, just split into bounded modules. Each module has its own controller, service, and module file, and modules aren't allowed to inject each other's services directly. We went with this mainly because we're a small team and the domain is pretty well understood at this stage, so going full microservices this early would just be extra operational overhead for not much gain. Since the modules are already cleanly separated, if we ever did need to split into microservices later, it wouldn't mean a big rewrite.

External Microservice (Messaging)

In-app messaging (UC4) runs through Firebase Firestore as its own separate microservice, kept outside the core monolith on purpose. The frontend talks to Firestore directly through the client SDK, so the NestJS backend isn't sitting in the middle of that traffic at all. That means if Firestore ever goes down, it doesn't take the rest of the platform with it. That's a graceful degradation boundary we built in deliberately, not something that just happened. The auction feature reuses the same boundary for its live bid updates: the backend writes the current highest bid to Firestore only after the Postgres transaction has committed, and the frontend listens to that document directly.

Stateless API Layer

The NestJS API doesn't keep any session state on the server side. Auth works through a JWT plus an HTTP-only cookie, and the cookie settings change depending on the environment (`sameSite: 'none'` and `secure: true` once it's deployed, since frontend and backend sit on different hostnames there, and `sameSite: 'lax'` locally where that isn't an issue). Because there's no server-side session state, any backend instance can handle any request, which is what actually makes horizontal scaling possible without needing sticky sessions.

Read-Heavy Performance Strategy

People browse and search listings way more than they create new ones, so the schema has indexes on the columns that get used most in filters and sorting: a composite index on `(module_id, price)`, plus separate indexes on `condition`, `annotation_level`, `status`, `seller_id`, `reviewed_by`, and `created_at`. Books get indexed on `isbn`, `(author, title)`, and `edition`. Audit log entries are indexed on `(entity_type, entity_id)` and

`performed_at`.

Repository Pattern (Data Access)
Services never write raw SQL directly. All database access goes through TypeORM repositories, so the actual Postgres queries stay abstracted away from the business logic. This is basically the object-persistence pattern we covered in the architecture lectures (L11), where a DB manager layer sits between the business objects and the actual database, so the business logic doesn't need to know or care what's underneath it.

Event-Driven Job Scheduling (Auctions)
Auctions have to open and close at an exact moment without a human doing anything. Instead of a cron job scanning the auction table every few seconds, each auction schedules delayed jobs on a Redis-backed queue (BullMQ): an activation job when the start time is in the future, and a close job for every auction. A job fires once, exactly when it is needed, so there is no wasted polling. Because a bid in the last 30 seconds extends the end time (anti-sniping), the close job is self-correcting. When it fires it re-reads the auction, and if the end time has moved it simply reschedules itself for the new time instead of closing. This is simpler and safer than cancelling and re-adding jobs, which would create a race between two extensions. Redis runs as a plain container inside the Container Apps Environment with internal ingress only, so it is not reachable from the internet.

Pessimistic Locking (Bid Placement)
Two students can bid at the same instant, so placing a bid runs inside one Postgres transaction that locks the auction row with `SELECT ... FOR UPDATE`. Concurrent bids are serialised, the second one is validated against the first one's result, and exactly one of two identical bids is accepted. The locked query deliberately has no joins, because Postgres cannot lock the nullable side of an outer join, and no external calls are made while the lock is held.

Graceful Degradation (AI-Assisted Listing Creation)
AI-assisted listing creation has two independent pipelines, and both are optional. The OCR pipeline is backend-mediated: the frontend sends the uploaded photo's URL to the Vision module, which calls Azure AI Vision, fuzzy-matches the text against the `Book` table and returns a suggestion. The crop pipeline runs entirely in the browser, using a dependency-free flood fill to remove uniform backgrounds and detect four corners, which the seller can drag before confirming. If the OCR call fails or times out, or the background is too busy to detect, the seller gets a short non-blocking message and manual entry works exactly as it always did.

2.2 Design Patterns

- **Decorator Pattern.** NestJS guards (`@UseGuards(JwtAuthGuard, RolesGuard)`) add auth checks onto a controller method without touching what the method actually does. Keeps the security stuff separate from the endpoint's actual job.
- **Builder Pattern.** TypeORM's `createQueryBuilder()` is used for listing search. We start with a base query and keep adding `andWhere()` clauses depending on which filters were actually sent in the request, instead of writing a separate hardcoded query for every possible combination of filters.
- **Factory Pattern.** `ListingResponseFactory` is a small factory that turns `Listing` entities into the objects the API actually returns, adding stuff like a formatted price or a status label that isn't stored in the DB itself. Keeps the response shape the same across every listing endpoint.
- **Producer-Consumer (Job Queue).** The Auction service produces delayed `activate-auction` and `close-auction` jobs, and a separate processor consumes them. Scheduling and execution are decoupled, and the processor can be tested on its own.
- **Observer.** Firestore listeners push new messages and new highest bids to connected clients, so the UI reacts to changes instead of polling for them.
- **Fuzzy Matching Utility.** `book-matcher.util.ts` scores OCR text against candidate books and only returns a match above a confidence threshold, keeping the messy matching logic out of the controller and the service.

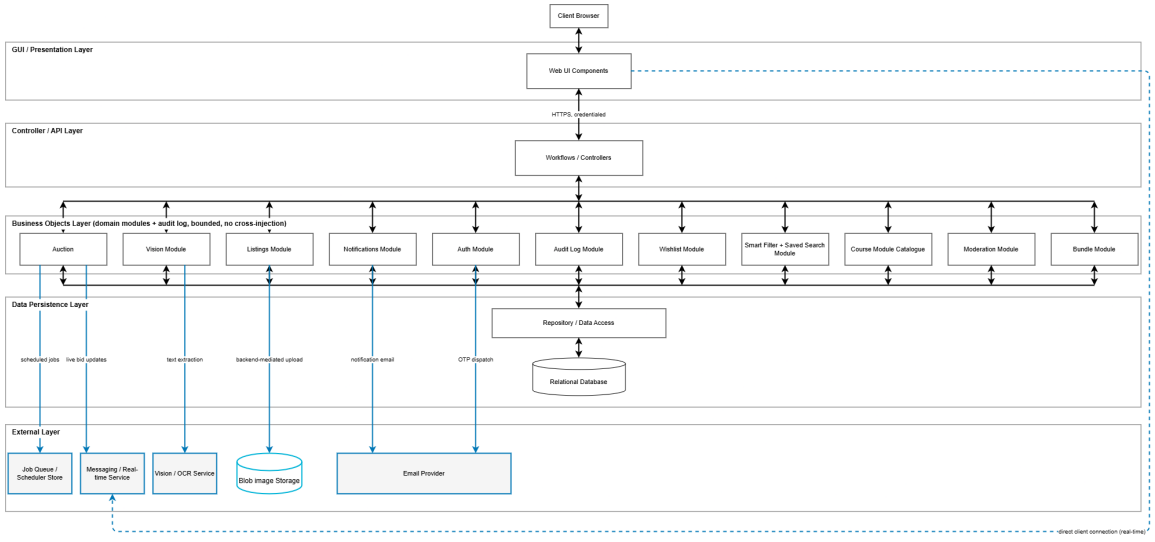
2.3 Constraints

Constraint	Detail
Hosting	Azure, under Agile Bridge's subscription (through our client contact Kyle). We manage the actual deployment but not the subscription/tenant itself.
Authentication	Custom OTP email verification, not a managed identity provider like Azure B2C.
Email allowlist	Student email domains of fifteen South African universities, with `@tuks.co.za` (University of Pretoria) as the primary domain in active use. The list is configurable through backend environment variables.
No payment processing	The platform only sets up meetup-based exchanges. No payment gateway, escrow, or any kind of financial transaction.

Constraint	Detail
ISBN metadata	No external ISBN lookup API. Sellers can type book details in, or photograph the cover and let OCR pre-fill them from books already in our own database.
AI service quota	Azure AI Vision runs on the free (F0) tier, which has low per-minute and monthly call limits. OCR is only called when a seller chooses to scan a cover, and automated tests use fixtures instead of the live service.
Redis availability	A single Redis container with no replication or failover, which is acceptable for a time-boxed project. If it is down, auctions cannot schedule or close, but the rest of the platform keeps working.
No mobile app	Web only for now. A React Native app is more of a stretch goal, not something we're actually building.
Multi-university support	The allowlist covers fifteen universities, but `tuks.co.za` is the only one in active use. The data model itself supports more than one university.

2.4 Architectural Diagram

This is a tech-neutral view of the main components in the system and how they talk to each other.



Notice how messaging doesn't go through the NestJS gateway at all, that's the graceful degradation boundary from §2.1, shown visually here. The diagram stays technology-neutral, so Redis appears as the job queue and scheduler store, and Azure AI Vision appears as the OCR service. Both sit in the external layer, reached only by the Auction and Vision

modules respectively.

2.5 Mapping Quality Requirements to Architectural Decisions

Quality Requirement	Architectural Decision
Fast multi-filter listing search	Composite and single-column Postgres indexes on the columns used for listing search and sorting. Building the query with the builder pattern avoids unnecessary full-table scans.
Core features stay up if messaging fails	Messaging is kept separate as an external Firestore microservice, hit directly by the frontend, so if Firestore goes down it can't cascade into Auth, Listings, or Moderation.
Horizontal scalability	Stateless JWT-based auth means any backend instance can serve any request. Azure Container Apps can just add replicas without needing sticky sessions.
Only verified students can create listings, only admins can moderate	OTP verification happens at registration. `JwtAuthGuard` and `RolesGuard` (Decorator pattern) sit on every protected route.
Modular, low-coupling codebase	NestJS bounded modules (Auth, Listings, Moderation, Reports and Bans, Modules, Saved Search, Wishlist, Notifications, Audit Log, Auction, Vision, Bundle), with no module directly injecting another module's service.
WCAG AA accessibility	Checked through automated Lighthouse audits. Currently around 95% AA compliant.
Measurable backend test coverage	`ci.yml` runs `npm run test:cov` against a real, temporary Postgres 15 container on every push and PR to `main/develop`. Coverage gets uploaded to Codecov.
Code quality and static analysis	SonarCloud runs on every push to `main/develop` and on every PR targeting either branch.
Reproducible, environment-parity deployment	Two fully separate Container App pairs for production and staging, each with its own database, both deployed automatically through SHA-tagged images (see §6).

Quality Requirement	Architectural Decision
Only one winning bid under simultaneous bidding	Bid placement runs in one Postgres transaction that locks the auction row (pessimistic locking), so concurrent bids are serialised and exactly one of two identical bids is accepted.
Auctions open and close on time without polling	Delayed jobs on a Redis-backed queue fire once at the exact time. The close job re-checks the end time when it fires, so an anti-sniping extension can never cause an early or double close.
AI-assisted listing creation can never block selling	OCR is an optional, time-limited, backend-mediated pre-fill, and the crop runs in the browser. If either fails, manual entry works exactly as before.
Cloud credentials never reach the client	The Vision endpoint and key, the storage credentials and the database connection string live only in the backend as Container Apps secrets.

3. Technology Requirements

Layer	Technology	Justification
Frontend	Next.js, React, TypeScript	Server-side rendering helps with how fast the listing browse pages feel, which are the pages that get the most traffic. TypeScript keeps the codebase type-safe throughout.
Frontend styling	Tailwind CSS	Utility-first styling means we can iterate faster, and the token-based theming keeps things consistent with the Agile Bridge brand style guide.
Backend	NestJS, TypeScript	The module structure lines up directly with our bounded-module architecture. Decorators cut down on repetitive guard and DTO code.
Database	Azure Database for PostgreSQL (Flexible Server)	Relational data suits our domain well, and it's fully ACID-compliant. Being Azure-native also fits our hosting constraint.
ORM	TypeORM	Gives us the repository pattern for data access, plus a migration system for changing the schema over time.
Authentication	Custom OTP plus JWT	Matches the client's OTP verification requirement. JWT is stateless, so we're not tied to a managed identity provider.
File storage	Azure Blob Storage, backend-mediated with SAS tokens	Uploads go through the backend first instead of straight to Blob Storage, so storage credentials never touch the client.
Messaging	Firebase Firestore	Handles real-time chat as its own external microservice. If it goes down, the rest of the platform keeps working. The SDK also handles real-time subscriptions on its own, no backend proxying needed. It also carries the live highest bid for each auction.

Layer	Technology	Justification
Job queue	Redis (`redis:7-alpine` container) with BullMQ, through `@nestjs/bullmq` and `ioredis`	Event-driven scheduling for auction activation and closing, with no polling. Running it as a plain container inside the existing Container Apps Environment kept it cheap, and it avoids the TLS configuration branch a managed Redis service would have needed.
AI text recognition	Azure AI Vision (Image Analysis 4.0 Read API, F0 tier)	Reads title, author and ISBN from a photographed cover. Called only from the backend, so the key never reaches the client.
Client-side image processing	Custom canvas flood fill (`removeUniformBackground.ts`)	Removes uniform backgrounds and detects book corners in the browser. We first tried an ONNX background-removal model, but its size of roughly 176MB made it too slow for phones and a cold staging site.
Email	Mailtrap (sandbox on staging, live SMTP on production)	Delivers OTP codes and notification emails through one provider class, with the sandbox or live choice made by the environment.
Custom domain and TLS	Registrar DNS (CNAME and TXT records) and an Azure-managed certificate	Serves the frontend on `www.unitextbookmarketplace.co.za` over HTTPS at no extra cost.
Hosting	Azure Container Apps	Container-native, and it has a built-in revision/rollback model that we've actually had to use, which is a big part of why we chose it over App Service.
Container Registry	Azure Container Registry	Private image storage. `az acr build` lets us build remotely without needing Docker running locally.

Layer	Technology	Justification
CI (lint, test, coverage)	GitHub Actions, `ci.yml`	Runs on every push and PR to `main`/`develop`. Backend tests run against a real, temporary Postgres container instead of a mock, so we trust the results more.
CD (build, deploy)	GitHub Actions, `deploy.yml`	Branch-conditional, `main` deploys to production and `develop` deploys to staging. Images are tagged with the commit SHA instead of `:latest`, so Container Apps always picks up a real change.
Code quality and static analysis	SonarCloud	Runs on every push to `main`/`develop` and on every PR targeting either branch.
Coverage reporting	Codecov	Separate `backend` and `frontend` flags per CI job.
Testing (backend)	Jest, plus a real temporary Postgres container in CI	Gives us integration-level confidence instead of just relying on a mocked database.
Testing (frontend)	Jest, React Testing Library and Cypress	Jest and React Testing Library cover components and utilities, and Cypress runs end-to-end specs in CI.
API documentation	OpenAPI (`docs/OpenAPI/openapi.yaml`) and Swagger, generated from NestJS decorators	The shared `openapi.yaml` is the contract-first source of truth, and Swagger gives interactive docs straight from the controller code.

4. API Contracts

All endpoints are prefixed with `/api`. Every authenticated endpoint needs the header `Authorization: Bearer <JWT>`. All request and response bodies use `Content-Type: application/json`. Every endpoint is also described in the shared `docs/OpenAPI/openapi.yaml`, which is version-controlled with the code.

4.1 Carried Over from Demo 1 (confirmed still in use)

4.2 Auth API Contracts

4.2.1 POST /auth/register

Description: Starts student registration. Checks the email domain and sends out the OTP.
Auth required: No

Request body:

```
{
  "fullName": "string (required)",
  "surname": "string (required)",
  "university": "string (required)",
  "email": "string (required, must be allowed domain)"
}
```

Success response (201 Created):

```
{
  "message": "Registration successful. Check your university email for
     verification code",
  "userId": "uuid"
}
```

Error responses:

- 400: Missing required field, or email domain not in allowlist
- 409: Email address is already registered

4.2.2 POST /auth/verify-otp

Description: Checks the 6-digit OTP sent to the student's email.
Auth required: No

Request body:

```
{
  "email": "string (required)",
  "otp": "string (required, 6 digits)"
}
```

Success response (200 OK):

```
{
  "message": "Email verified successfully.",
  "verified": true
}
```

Error responses:

- 400: OTP is incorrect or has expired
- 404: No pending OTP found for this email

4.2.3 POST /auth/login

Description: Logs in a verified student and hands back a signed JWT.
Auth required: No

Request body:

```
{
  "email": "string (required)",
  "password": "string (required, min 8 characters)"
}
```

Success response (200 OK):

```
{
  "message" : "Login sucessful."
}
```

Error responses:

- 401: Invalid email or password
- 403: Account exists but email is not verified

4.2.4 POST /auth/set-password

Description: Sets the student's password once OTP verification is done.
Auth required: No (uses `userId` from the registration step)

Request body:

```
{
  "userId": "uuid (required)",
  "password": "string (required, min 8 characters)",
  "confirmPassword": "string (required, must match password)"
}
```

Success response (200 OK):

```
{
  "message": "Password set successfully.",
  "accessToken": "string (JWT)"
}
```

4.2.5 GET /auth/me

Description: Returns the logged-in user's profile.
Auth required: Yes

4.2.6 POST /auth/logout

Description: Clears the session cookie.
Auth required: Yes

4.2.7 GET /auth/universities

Description: Lists supported universities, for the registration dropdown.
Auth required: No

4.2.8 POST /auth/refresh

Description: Gets a new access token using a refresh token, so users stay logged in without having to type their password again.
Auth required: No

4.3 Listings API Contracts

4.3.1 POST /listings

Description: Creates a new textbook listing. Gets created with `status = PENDING`.
Auth required: Yes (Student role)

Request body:

```
{
  "isbn": "string (optional)",
  "title": "string (required)",
  "author": "string (optional)",
  "edition": "integer (required)",
  "condition": "new | good | fair | poor (required)",
  "annotationLevel": "none | light | heavy (required)",
  "bookId": "uuid (required)",
  "moduleId": "uuid (required)",
  "price": "number, positive decimal (required)",
  "hasNotes": "boolean (required)",
  "photoUrls": ["string (required >= 4)"],
  "description: "string (required)"
}
```

4.3.2 GET /listings

Description: Returns all APPROVED listings, with pagination and filters.
Auth required: Yes
Query parameters: `moduleId`, `faculty`, `condition`, `minPrice`, `maxPrice`, `edition`, `page`, `limit`

4.3.3 GET /listings/mine

Description: Returns all of the logged-in student's own listings, including PENDING, APPROVED and REJECTED ones.
Auth required: Yes (Student role)

4.3.4 GET /listings/:id

Description: Returns the full detail object for one listing.
Auth required: Yes
Path parameter: `id`, the UUID of the listing

4.4 Admin API Contracts

4.4.1 GET /listings/admin/all

Description: Returns all listings, admin only.
Auth required: Yes (Admin role only)

4.4.2 PATCH /admin/listings/:id/approve

Description: Approves a pending listing. Sets status to APPROVED and writes an audit log entry.
Auth required: Yes (Admin role only)

4.4.3 PATCH /admin/listings/:id/reject

Description: Rejects a pending listing. Sets status to REJECTED and writes an audit log entry, a reason is included with the rejection.
Auth required: Yes (Admin role only)

Note: The audit log write was added for Demo 2

4.5 Modules API Contract

4.5.1 POST /modules

Description: Creates a module linked to the textbook being sold. Takes module details like name, code, semester, and faculty id.

4.5.2 GET /modules/faculties

Description: Returns faculty names and ids. Used when creating a module during listing creation

4.6 New for Demo 2

4.7 Listing API Contract

4.7.1 PATCH /listings/editlist

Module: Listings (UC5)
Description: Edits the details of an existing listing that the student owns. Only works on listings that are rejected or pending.
Auth required: Yes (Student role, listing owner)
Request body: Only listing details can be edited, everything is optional except the listing id.

```
{
 "id" : "string (uuid)",
 "title" : "string",
 "condition" : 'new' | 'good' | 'fair' | 'poor',
```

```
 "annotation_level" : 'none' | 'light' | 'heavy',
 "price" : "number",
 "has_notes" : "boolean",
 "description" : "string",
}
```

4.8 Module API Contract

4.8.1 GET /modules (extended filters)

Module: Smart Filters (UC6)
Description: Extends the existing module search endpoint with extra smart filter parameters.
Description: Search-as-you-type module code lookup. Returns modules matching the search query.
Auth required: Yes
Query parameters:

- `search` (string, required): partial module code or name to search
- `university` (string, optional): filter by university name (e.g. `"UP"`)

4.9 Saved Searches API Contract

4.9.1 POST /saved-searches/

Module: Saved Search (UC6-ext)
Description: Saves a search as JSON **Request body:** all fields are optional

```
{
 "filter_json" : {
 "faculty" : "string",
 "moduleCode" : "string",
 "edition" : "string",
 "priceMin : "string | number",
 "priceMax" : "string | number",
 "condition" : "string",
 "annotationLevel" : "string",
 "search" : "string",
 "book_title" : "string",
 "author" : "string",
 "isbn" : "string",
 "university_id" : "string",
```

```
 "faculty_id": "string"
 }
}
```

4.9.2 GET /saved-searches/me

Module: Saved Search (UC6-ext)
Description: Returns the saved searches belonging to the logged-in student. Confirmed to exist as a route in `saved_search.controller.ts`.
Auth required: Yes
Request body: uses query parameters.

- `id` (string)
- `email` (string)
- `role` (string)
- `firstname` (string)
- `lastname` (string)

4.9.3 GET /saved-searches/id

Module: Saved Search (UC6-ext)
Description: Returns one saved search by id **Auth required:** Yes
Request body: uses query parameters.

- `id` (string)

4.9.4 DELETE /saved-searches/id

Module: Saved Search (UC6-ext)
Description: Deletes a saved search by id **Auth required:** Yes
Request body: uses query parameters.

- `id` (string)

4.9.5 GET /saved-searches/me/email

Module: Saved Search
Description: Returns the logged-in student's saved search email.

4.9.6 GET /saved-searches/me/role

Module: Saved Search
Description: Returns role-specific saved search settings for the logged-in user.

4.10 Wishlist API Contract

4.10.1 POST /wishlist/:listingId

Module: Wishlist (UC7)
Description: Adds a listing to the logged-in student's wishlist.
Auth required: Yes (Student role)
Request body:

```
{
 "userid" : "string (uuid)",
 "listingid" : "string(uuid)",
}
```

4.10.2 DELETE /wishlist/:listingId

Module: Wishlist (UC7)
Description: Removes a listing from the logged-in student's wishlist.
Auth required: Yes (Student role)
Request body:

```
{
 "userid" : "string (uuid)",
 "listingid" : "string(uuid)",
}
```

4.10.3 GET /wishlist/mine

Module: Wishlist (UC7)
Description: Returns the logged-in student's wishlisted listings.
Auth required: Yes (Student role)

4.11 Admin API Contract

4.11.1 GET /admin/audit-log

Module: Audit Log (UC8)
Description: Returns audit log entries for admin review.
Auth required: Yes (Admin role only)
Query parameters: all are optional

- `performedBy` (string): filter by which admin did the action
- `action` (string): approved a listing or rejected a listing
- `entityType` (string): what type the action was performed on, currently only listing
- `entityId` (string)
- `startDate` (string)
- `endDate` (string)

4.11.2 GET /admin/emails

Module: Audit Log (UC8)
Description: Returns a list of admin emails, used for filters **Auth required:** Yes (Admin role only)

4.12 Image API Contract

4.12.1 POST /images/upload

Module: Blob upload
Description: Uploads a listing image to Azure Blob Storage through the backend.
Auth required: Yes (Student role)

4.12.2 GET /images/list

Module: Blob upload
Description: Lists uploaded images for a listing.
Auth required: Yes

4.12.3 DELETE /images/delete

Module: Blob upload
Description: Deletes an uploaded listing image from Blob Storage.

Auth required: Yes (Student role, listing owner)

4.13 Conversations API Contract

4.13.1 POST /conversations

Module: Messaging
Description: Creates or fetches a conversation for a listing
Auth required: Yes
Request body:

```
{
  "listingId": "uuid"
}
```

4.13.2 POST /conversations/mine

Module: Messaging
Description: Gets all conversations for the logged-in user
Auth required: Yes

4.13.3 POST /conversations/:id/messages

Module: Messaging
Description: Gets all messages in a conversation
Auth required: Yes
Api param:

```
{
  name: 'id',
  description: 'Conversation ID',
}
```

4.13.4 GET /conversations/:id/messages

Module: Messaging
Description: Sends a message in a conversation
Auth required: Yes
Api param:

```
{
```

```
  name: 'id',
  description: 'Conversation ID',
}
```

Request body:

```
{
  "text" : "string"
}
```

4.14 Demo 3 API Contracts:

4.14.1 POST /reports

Module: Reports **Description:** Creates a new report **Auth required:** Yes **Request body:**

```
{
"listing_id": "uuid",
"reason": "Fraud | Misleading | Duplicate | Other"
}
```

4.14.2 GET /admin/reports

Module: Reports / Admin **Description:** Getter for reports. **Auth required:** Admin only **Query parameters:** Optional report filters.

Module: Reports / Admin **Description:** Specific getter for reports review. **Auth required:** Admin only

4.15 Admin Moderation API Contract

4.15.1 PATCH /admin/reports//dismiss

Module: Admin / Reports **Description:** Dismisses a report **Auth required:** Admin only

4.15.2 PATCH /admin//ban

Module: Admin / Moderation **Description:** Perform the ban on a report **Auth required:** Admin only **Request body:**

```
{
"reason": "string"
```

```
}
```

4.16 Notifications API Contract

4.16.1 GET /notifications/mine

Module: Notifications
Description: Returns the logged-in user's notifications, paginated.
Auth required: Yes
Query parameters:

- `page` (number, optional, default 1)
- `limit` (number, optional, default 10)

4.16.2 PATCH /notifications/:id/read

Module: Notifications
Description: Marks a single notification as read.
Auth required: Yes (owner only)
Path parameters:

- `id` (string, uuid): notification id

4.16.3 PATCH /notifications/read-all

Module: Notifications
Description: Marks all of the logged-in user's notifications as read.
Auth required: Yes

4.16.4 DELETE /notifications/:id/delete

Module: Notifications
Description: Deletes a notification belonging to the logged-in user.
Auth required: Yes (owner only)
Path parameters:

- `id` (string, uuid): notification id

4.17 Listing Status API Contract

4.17.1 PATCH /listings/:id/status

Module: Listings (UC5.2)
Description: Lets the seller move one of their approved listings through its sale lifecycle. `RESERVED` was added in Demo 3 next to the existing `SOLD` and `WITHDRAWN` values. The seller can trigger it from the messaging thread once a sale is agreed, or from My Listings, and the listing detail page shows the matching badge.
Auth required: Yes (Student role, listing owner)
Path parameter: `id`, the UUID of the listing

Request body:

```
{
  "status": "RESERVED | SOLD | WITHDRAWN (required)"
}
```

Error responses:

- 400 or 403: The transition is not allowed from the listing's current status, or the caller does not own the listing
- 401: Missing or invalid JWT

4.18 Demo 4 API Contracts

Vision API Contract (Tiego Mokwena)

4.18.1 POST /vision/extract-text

Module: Vision (AI-Assisted Listing Creation, Wow Factor 1)
Description: Runs text recognition on a book cover photo that has already been uploaded through the existing image flow, then fuzzy-matches the extracted text against the `Book` table. The Azure AI Vision endpoint and key stay in the backend and never reach the client. The result is a suggestion only. The frontend uses it to pre-fill title, author, ISBN, edition and book id, and the seller can edit every field.
Auth required: Yes (Student role)

Request body:

```
{
  "imageUrl": "string (required, URL of the uploaded cover photo)"
}
```

Success response:

```
{
```

```
  "rawText": "string",
  "matchedBook": {
    "id": "uuid",
    "title": "string",
    "author": "string",
    "isbn": "string",
    "edition": "integer",
    "confidence": "number, 0 to 1"
  } | null
}
```

Notes:

- `matchedBook` is `null` when no candidate book scores above the confidence threshold. `rawText` is still returned so the frontend can show something.
- The frontend aborts the call after about 10 seconds. On any failure or timeout it shows a non-blocking message and keeps the cropped photo, so manual entry is never blocked.
- Background removal and the draggable corner crop run entirely in the browser, so there is no second endpoint for them.

Error responses:

- 400: `imageUrl` missing or not a valid URL
- 401: Missing or invalid JWT
- 403: Caller does not have the Student role
- 5xx: The Azure AI Vision call failed. The frontend falls back to manual entry.

Auction API Contract (Gift Mohuba)

Overview. All auction endpoints are served under `/auction` and require a valid JWT. Auction status values are `SCHEDULED`, `ACTIVE`, `ENDED` and `CANCELLED`. The minimum bid increment is R5. Live state is mirrored to the Firestore document `actions/{auctionId}` so that clients update without polling (see the notes on `POST /auction/:id/bid`).

4.18.2 POST /auction

Module: Auctions
Description: Creates an auction for one of the caller's approved listings. If `start_time` is omitted or is not in the future, the auction starts immediately with status `ACTIVE`. Otherwise it is created as `SCHEDULED` and activated by a background job at the start time.

A second background job closes the auction at `end_time`.
Auth required: Yes (Student role, listing owner)

Request body:

```
{
  "listing_id": "uuid (required)",
  "starting_price": "number (required)",
  "reserve_price": "number (optional)",
  "start_time": "ISO 8601 date-time (optional)",
  "end_time": "ISO 8601 date-time (required)"
}
```

Success response (201):

```
{
  "message": "Auction created successfully"
}
```

Notes:

- A listing can only have one `ACTIVE` or `SCHEDULED` auction at a time.
- The listing must be approved and must not be `SOLD` or `RESERVED`.
- The reserve price is optional. When it is not set, the starting price is used as the reserve when the auction closes.

Error responses:

- 400: The listing is not approved, is sold or reserved, already has an active or scheduled auction, or `start_time` is not before `end_time`
- 401: Missing or invalid JWT
- 403: The listing does not exist, or the caller does not own it, or the caller does not have the Student role
- 500: The closing or activation jobs could not be scheduled. The auction is removed and nothing is created.

4.18.3 GET /auction

Module: Auctions
Description: Returns all auctions, each with its listing (book, module, faculty, university and seller) and the total number of bids placed on it.
Auth required: Yes

Success response (200):

```
[
  {
    "id": "uuid",
    "listing_id": "uuid",
    "listing": { ... },
    "seller_id": "uuid",
    "starting_price": "string (decimal)",
    "current_highest_bid": "string (decimal) | null",
    "current_highest_bidder_id": "uuid | null",
    "start_time": "ISO 8601 date-time",
    "end_time": "ISO 8601 date-time",
    "status": "SCHEDULED | ACTIVE | ENDED | CANCELLED",
    "extension_count": "integer",
    "created_at": "ISO 8601 date-time",
    "bidCount": "integer"
  }
]
```

Error responses:

- 401: Missing or invalid JWT

4.18.4 GET /auction/:id

Module: Auctions
Description: Returns the full current state of one auction, including its listing, seller and current highest bidder.
Auth required: Yes
Path parameters:

- `id` (string, uuid): auction id

Success response (200): A single auction object with the same fields as the items returned by `GET /auction`, without `bidCount`.

Error responses:

- 401: Missing or invalid JWT
- 404: Auction not found

4.18.5 GET /auction/listing/:listingId

Module: Auctions
Description: Returns the `ACTIVE` or `SCHEDULED` auction for a listing, or `null` when the listing has none. Used by the listing page to show an auction link.
Auth required: Yes
Path parameters:

- `listingId` (string, uuid): listing id

Success response (200): An auction object as described for `GET /auction/:id`, or `null`.

Error responses:

- 401: Missing or invalid JWT

4.18.6 POST /auction/:id/bid

Module: Auctions
Description: Places a bid on an auction. The auction row is locked inside a database transaction, so two simultaneous bids are processed one after the other. Every attempt is stored, either as `ACCEPTED` or as `REJECTED` with a reason.
Auth required: Yes (Student role)
Path parameters:

- `id` (string, uuid): auction id

Request body:

```
{
  "amount": "number (required)"
}
```

Success response (201), bid accepted:

```
{
  "status": "ACCEPTED",
  "auction": { ... updated auction ... }
}
```

Success response (201), bid rejected by a validation rule:

```
{
  "status": "REJECTED",
  "reason": "string"
}
```

Bid validation rules. A bid is rejected, with the reason shown in brackets, when:

- the auction is not `ACTIVE` (`auction not active`)
- the end time has passed (`auction has ended`)
- the bidder is the seller (`seller cannot bid on own auction`)
- the bidder already holds the highest bid (`already the highest bidder`)
- the amount is below the minimum, which is the starting price for the first bid and the current highest bid plus R5 afterwards (`bid must be at least <amount>`)

Anti-sniping behaviour. If an accepted bid arrives when fewer than 30 seconds remain, the end time is moved to 30 seconds after that bid and `extension_count` is increased by one. The closing job re-checks the end time and re-schedules itself when the auction has been extended.

Real-time update. After an accepted bid, the backend updates the Firestore document `actions/{auctionId}` with `currentHighestBid`, `currentHighestBidderName`, `endTime` and `extensionCount`. Clients listen to this document and refresh the bid history when it changes.

Error responses:

- 401: Missing or invalid JWT
- 403: Caller does not have the Student role
- 404: Auction not found

4.18.7 GET /auction/:id/bids

Module: Auctions
Description: Returns the paginated bid history of an auction, newest first.
Auth required: Yes
Path parameters:

- `id` (string, uuid): auction id

Query parameters:

- `page` (number, optional, default 1)
- `limit` (number, optional, default 20, maximum 100)

Success response (200):

```
{
  "data": [
```

```
    {
      "id": "uuid",
      "amount": "string (decimal)",
      "status": "ACCEPTED | REJECTED",
      "rejection_reason": "string | null",
      "placed_at": "ISO 8601 date-time",
      "bidder": { "first_name": "string", "last_name": "string", ... }
    }
  ],
  "meta": {
    "total": "integer",
    "page": "integer",
    "limit": "integer",
    "totalPages": "integer"
  }
}
```

Error responses:

- 400: `page` or `limit` is not an integer
- 401: Missing or invalid JWT

4.18.8 Auction closing (background job)

Module: Auctions
Description: Not an HTTP endpoint. A queued job runs at `end_time` and sets the auction status to `ENDED`. The outcome is `SOLD` when the highest bid is at least the reserve price (or the starting price when no reserve was set), `RESERVE_NOT_MET` when bids exist but none reached it, and `NO_BIDS` when nobody bid. The outcome, winner id and end time are written to `actions/{auctionId}`. On a sale, the listing is marked `RESERVED`, a conversation between seller and winner is created, and both parties are notified.

4.18.9 POST /bundles/optimize

Module: Bundles
Description: Given the modules a student is taking, works out which textbooks are required and builds the cheapest way to buy them. Returns two options for comparison: an *optimized bundle*, which groups purchases by seller (greedy selection favouring sellers who cover the most uncovered books per unit of price) to keep the number of sellers low, and the *cheapest individual option*, which picks the lowest priced listing for each book regardless of seller. Only listings with status `APPROVED` are considered, and a book shared

by several modules is only counted once.

Auth required: Yes (JWT bearer token, `Authorization: Bearer <token>`)

Request body:

```
{
    "moduleIds": ["<module-uuid>", "<module-uuid>"]
}
```

Field	Type	Description
`moduleIds`	`string[]` (UUIDs)	Modules the buyer needs books for. Must contain at least one module id.

Success response (200 OK):

```
{
    "requiredBooks": [ /* ModuleBook objects, with book and module */ ],
    "approvedListings": [ /* Listing objects, with book and seller */ ],
    "optimizedBundle": {
        "listings": [ /* Listing objects */ ],
        "sellerIds": ["<seller-uuid>"],
        "totalPrice": 850.00,
        "booksCovered": 4
    },
    "cheapestIndividualOption": {
        "listings": [ /* Listing objects */ ],
        "sellerIds": ["<seller-uuid>", "<seller-uuid>"],
        "totalPrice": 790.00,
        "booksCovered": 4
    }
}
```

Field	Description
`requiredBooks`	Unique books required across the selected modules (one `ModuleBook` per book).
`approvedListings`	Every approved listing for the required books.
`optimizedBundle.listings`	Listings chosen by the seller-grouping algorithm.
`optimizedBundle.sellerIds`	Sellers the buyer would purchase from, in selection order.
`optimizedBundle.totalPrice`	Sum of the prices of the selected listings.
`optimizedBundle.booksCovered`	Number of required books covered. Lower than the number of required books if some books have no approved listing.
`cheapestIndividualOption.*`	Same shape as `optimizedBundle`; `sellerIds` contains the distinct sellers of the cheapest listings.

Error responses:

Status	Reason
`400 Bad Request`	`moduleIds` is empty ("At least one module must be selected").
`401 Unauthorized`	Missing or invalid JWT.

Cases and Appeals API Contract (Neo Bosoga)

Overview: A *case* is created when a banned user appeals their ban. An admin reviews the case and either upholds the ban or reverses it (which unbans the user). Cases are soft-deleted (`deleted_at`), never removed from the database. All endpoints below are under the `/cases` prefix and require a valid JWT (`Authorization: Bearer <token>`). Endpoints marked *admin only* additionally require the `admin` role and return `403 Forbidden` otherwise.

Case object (CaseResponse)

Field	Type	Description
`id`	uuid	Unique identifier of the case.
`user_id`	uuid	The user who submitted the appeal.
`appeal_message`	string, nullable	The appeal message from the user.
`status`	enum	`pending` (awaiting admin review), `upheld` (ban maintained), `reversed` (ban lifted, user reinstated).
`reviewed_by`	uuid, nullable	The admin who reviewed the case.
`reviewed_at`	date-time, nullable	When the case was reviewed.
`created_at`	date-time	When the case was created. Used for ordering and for tracking stale cases.
`updated_at`	date-time, nullable	When the case was last updated.
`deleted_at`	date-time, nullable	When the case was soft-deleted.

Paginated response (PaginatedCasesResponse)

```
{
    "data": [ /* array of Case objects */ ],
    "meta": {
        "total": 42,
        "page": 1,
        "limit": 20,
        "totalPages": 3
    }
}
```

Field	Type	Description
`data`	Case[]	Cases on the current page.
`meta.total`	integer	Total number of cases.
`meta.page`	integer	Current page number.
`meta.limit`	integer	Items per page.
`meta.totalPages`	integer	Total number of pages.

4.18.10 POST /cases

Module: Cases

Description: Submits an appeal as a banned user. Only users with `is_banned = true` can submit an appeal, and a user cannot have more than one pending appeal. Creates a case with status `pending`.

Auth required: Yes (banned user)
Request body:

```
{
    "appeal_message": "I believe my account was banned in error because..."
}
```

Success response: `201 Created`, returns a Case object.
Error responses: `400` user is not banned or already has a pending appeal, `401` not authenticated.

4.18.11 GET /cases/mine

Module: Cases
Description: Returns all cases submitted by the authenticated user.
Auth required: Yes
Success response: `200 OK`, returns an array of Case objects.
Error responses: `401` not authenticated.

4.18.12 GET /cases/admin

Module: Cases (Admin)
Description: Returns all cases with pagination for the admin dashboard.
Auth required: Yes (admin only)
Query parameters (all optional):

Parameter	Type	Description
`page`	integer	Page number (default 1).
`limit`	integer	Items per page (default 20, max 100).
`status`	string	Filter by `pending`, `upheld` or `reversed`.
`search`	string	Search within the appeal message.

Success response: `200 OK`, returns a PaginatedCasesResponse.
Error responses: `401` not authenticated, `403` admin access required.

4.18.13 GET /cases/admin/pending

Module: Cases (Admin)
Description: Returns all cases with status `pending` for admin review (not paginated).
Auth required: Yes (admin only)
Success response: `200 OK`, returns an array of Case objects.
Error responses: `401` not authenticated, `403` admin access required.

4.18.14 GET /cases/:id

Module: Cases
Description: Returns a specific case by id. Users can only access their own cases.
Auth required: Yes
Path parameter: `id` (uuid), the case to retrieve.
Success response: `200 OK`, returns a Case object.
Error responses: `401` not authenticated, `404` case not found.

4.18.15 PATCH /cases/:id/review

Module: Cases (Admin)
Description: Records the admin's decision on a case. `upheld` keeps the user banned; `reversed` lifts the ban and the user is unbanned. Sets `reviewed_by` to the admin's id and updates `reviewed_at` and `updated_at`.
Auth required: Yes (admin only)
Path parameter: `id` (uuid), the case to review.
Request body:

```
{
    "decision": "upheld",
    "adminNotes": "Optional notes about the decision"
}
```

Field	Type	Description
`decision`	enum (required)	`upheld` or `reversed`.
`adminNotes`	string (optional)	Notes from the reviewing admin.

Success response: `200 OK`, returns the updated Case object.
Error responses: `400` invalid decision or case already reviewed, `401` not authenticated, `403` admin access required, `404` case not found.

5. Non-Functional Requirements (NFR) Testing

5.1 Testing Overview

Every quality requirement in the SRS has been quantified and verified with at least one executable, repeatable test or a documented inspection. Automated tests were run against the deployed staging environment (Azure Container Apps, South Africa North) unless stated otherwise, and each result has a saved artifact under `test/nfr/artifacts/`. Where the original requirement wording did not match the implemented architecture, the requirement has been reconciled with the implementation and the change is recorded in the relevant results subsection. The table below maps each NFR to its architectural tactic and the test that validates it.

5.2 NFR Traceability Matrix

ID	Requirement	Tactic in SAS	Test / Tool	Target	Status
NFR-01	GET /listings search returns within 2s at p95 (successful responses) under concurrent load on staging	Composite indexing, Query optimization, Pagination, Redis caching	k6	p(95) < 2000ms	PASS
NFR-02	New-notification delivery via the bell icon reflects a triggering event within 2 seconds (SSE-based)	Server-Sent Events push (GET /notifications/stream)	Playwright	< 2000ms	PASS
NFR-03	Stateless JWT-based API layer supports horizontal scaling without code changes	No session state on server, JWT tokens	Architecture Review	Pass	PASS
NFR-04	Production runs with min-replicas: 1, max-replicas: 2 on Azure Container Apps	Container orchestration, Auto-scaling	Azure CLI	Configured	PASS

ID	Requirement	Tactic in SAS	Test / Tool	Target	Status
NFR-05	Staging scales to zero when idle (min-replicas: 0) with cold-start delay 20s	Zero-scaling configuration	Azure CLI	< 30s	PASS
NFR-06	All protected endpoints enforce JWT-based authentication	JWT middleware, Token validation	Playwright	401 without token	PASS
NFR-07	Admin-only endpoints return 403 Forbidden for Student-role JWT	Role-based access control (RBAC)	Playwright	403 Forbidden	PASS
NFR-08	Passwords hashed with bcrypt at cost factor 12	bcrypt hashing, Salt generation	Static inspection	Cost factor 12	PASS
NFR-09	Authentication endpoints rate-limited per endpoint (login 10/min; register and verify-email 5/min; forgot-password and resend-otp 4/min)	Per-endpoint throttling (@Throttle)	Playwright	429 after limit exceeded	PASS
NFR-10	Access tokens expire after 15 minutes, refresh tokens after 7 days	JWT expiry configuration	Playwright (decoded JWT)	900s / 604800s	PASS
NFR-11	Banned user (is_banned=true) flagged in the login response (soft-ban) so the frontend can deny access to protected routes	User status check in AuthService.login	Playwright	200 with is_banned: true	PASS

ID	Requirement	Tactic in SAS	Test / Tool	Target	Status
NFR-12	Core features remain available if the external messaging microservice (Firestore) is unavailable	Isolation of the messaging dependency, Dependency injection	Static audit + core-features.spec.ts	Core features do not depend on Firestore	PASS
NFR-13	Production maintains at least 1 warm replica at all times	Minimum replica configuration	Azure CLI	1 replica	PASS
NFR-15	Backend test coverage measured on every push/PR via CI against an ephemeral PostgreSQL container, tracked in Codecov; merges gated on tests and SonarCloud Quality Gate (80% is a working target, not a hard gate)	CI/CD pipeline, Codecov integration	CI inspection, Codecov	Coverage measured; 80% target	PASS
NFR-16	Every new endpoint has a corresponding entry in openapi.yaml the same day it is committed	API documentation, OpenAPI spec	Manual Review	100% documented	PASS
NFR-17	UI meets Google Lighthouse Accessibility score 85/100 on every audited page	Semantic markup, ARIA labels	Lighthouse	85/100	PASS
NFR-18	Pages introduced from Demo 3 onward score 90/100 on Lighthouse	Accessibility-first design	Lighthouse	90/100	PASS

ID	Requirement	Tactic in SAS	Test / Tool	Target	Status
NFR-19	Every moderation, ban and appeal-decision action writes exactly one immutable audit_log entry	Audit log on admin actions, no update path, DB CHECK constraint	Integration tests (Jest)	1 row per action	PARTIAL
NFR-20	Audit log entries record action, admin UUID and timestamp	Structured audit records	Integration tests (Jest)	All fields present	PARTIAL

5.3 Test Results Summary

Category	Total	PASS	PARTIAL	FAIL	UNTESTED
Performance	2	2	0	0	0
Scalability	3	3	0	0	0
Security	6	6	0	0	0
Reliability/Availability	2	2	0	0	0
Maintainability	2	2	0	0	0
Accessibility	2	2	0	0	0
Auditability	2	0	2	0	0
TOTAL	**19**	**17**	**2**	**0**	**0**

5.4 Performance Results

5.4.1 NFR-01: Listing Search Latency

Requirement: Listing search shall respond with a p95 under 2000 ms.
Command:

```
k6 run -e BASE_URL=<staging-frontend-url> \
  --summary-export=test/nfr/artifacts/listing-load-summary.json \
  test/nfr/performance/listing-load.js \
  > test/nfr/artifacts/listing-load-summary.txt 2>&1
```

Method: `GET /api/listings?q=<query>&limit=20&offset=0` was tested against staging with 10 concurrent virtual users over 90 seconds (ramp-up, steady, ramp-down). Five realistic search terms were randomised per iteration.

Metric	Value
p95 of successful responses	480.88 ms (threshold < 2000 ms)
Real failures (non-200, non-429)	0.00%
Throttled (HTTP 429)	575 requests
Verdict	PASS

Note on throttling: The staging backend enforces a documented rate limit of 100 requests per 60 seconds per client IP. Under a 10-VU load, requests above that ceiling receive `429`. This is expected behaviour and not a defect; the latency figure is calculated from the successful responses only.
Evidence: `test/nfr/artifacts/listing-load-summary.json`

5.4.2 NFR-02: Notification Delivery Latency

Requirement (reworded): New-notification delivery via the bell icon shall reflect a triggering event within 2 seconds (SSE-based).
Reason for rewording: The original requirement referred to a fixed 15-second polling interval. The notification system uses Server-Sent Events (`GET /api/notifications/stream`), which is push-based: once a notification is persisted, the server publishes it to any open streams for that user. There is no polling interval in the current architecture, so the 15-second figure came from a legacy design. The 2000 ms threshold is strictly stronger than the original requirement, loose enough to absorb network, database commit and rendering latency, and tight enough to fail if the SSE path ever regresses.
Command:

```
npx playwright test --config=test/nfr/performance/playwright.config.ts
```

Method: Automated end-to-end test using Playwright's API request context (`test/nfr/performance/b`

1. Log in as the recipient and capture the auth cookies.
2. Open the SSE stream at `GET /api/notifications/stream` with the recipient's cookies.
3. Log in as a second user.
4. Trigger a notification by creating a conversation (`POST /api/conversations`) on an APPROVED listing owned by the recipient. This fires `notification.created`, which `NotificationStreamService` publishes to the recipient's open stream.
5. Measure the time from the trigger request (t0) to the arrival of the `notification.created` event on the stream.

Metric	Value
Threshold	2000 ms
Measured	995 ms
Environment	Staging (Azure Container Apps)
Verdict	PASS

Evidence: `test/nfr/artifacts/bell-delivery-summary.json` (includes the raw SSE frame captured during the test) and `test/nfr/artifacts/bell-delivery-playwright.json`.

5.5 Security Results

Command:

```
npx playwright test --config=test/nfr/security/playwright.config.ts
```

Requirement (reconciled with the implementation): Protected endpoints shall enforce JWT-based authentication and role-based access control. An admin-only endpoint shall return `403 Forbidden` for a Student-role JWT. Passwords shall be hashed with bcrypt at a cost factor of 12. Authentication endpoints shall be rate-limited per endpoint (login 10/min; register and verify-email 5/min; forgot-password and resend-otp 4/min). Access tokens shall expire after 15 minutes and refresh tokens after 7 days. A banned user (`is_banned = true`) shall be flagged in the login response with `is_banned: true` and `ban_reason`, so the frontend can deny access to protected routes; the API continues to issue tokens to allow ban-screen rendering and account-recovery flows.

Two clauses were reworded to match the implementation:

- **Ban enforcement is a soft ban, not a hard ban.** `AuthService.login` inspects `is_banned` and returns `200` with a flag instead of rejecting with `403`. This is a deliberate product decision so the frontend can show a ban notification and offer account-recovery routes.
- **Rate limiting is per endpoint.** Limits are applied with `@Throttle` decorators rather than a uniform 5/min across all authentication routes.

Method: Automated API test suite (`test/nfr/security/auth.spec.ts`) using Playwright's request context, run against the deployed staging environment. Six clauses, each with independent evidence.

#	Clause	Result	Evidence
1	Protected endpoints enforce JWT (NFR-06)	PASS	`GET /notifications/mine` without a token returns 401

#	Clause	Result	Evidence
2	Admin-only endpoint returns 403 for Student JWT (NFR-07)	PASS	`PATCH /admin/:id/approve` with a student cookie returns 403
3	Passwords hashed with bcrypt, cost factor 12 (NFR-08)	PASS	Static inspection: `BCRYPT_ROUNDS = 12` in `AuthService`
4	Authentication endpoints rate-limited per `@Throttle` config (NFR-09)	PASS	429 observed after rapid `POST /auth/login` requests
5	Access tokens expire in 15 min, refresh tokens in 7 days (NFR-10)	PASS	Decoded JWT: exp - iat = 900 s and 604800 s
6	Banned user flagged on login, soft ban (NFR-11)	PASS	`POST /auth/login` returns 200 with `is_banned: true` in the body

Overall result: 6/6 clauses verified on staging.
Evidence: `test/nfr/artifacts/security-summary.json` (machine-readable result, per-clause evidence, and notes on the two reworded interpretations).

5.6 Reliability and Availability Results

5.6.1 NFR-12: Core Features Independent of the Messaging Service

Requirement: Core features shall remain available if the external messaging microservice (Firestore) is unavailable.
Method: A static dependency audit combined with the automated specification `test/nfr/reliability/` Firestore is only reachable through the `firebase-admin` SDK via the `db` object exported from `firebase-admin.ts`. The audit shows that only `messaging.service.ts` imports that file.
Why core features cannot reach Firestore indirectly: NestJS dependency injection means the Firestore handle cannot flow into a controller or service that does not declare it in its constructor, and the compiler and module system enforce this. A core-feature request therefore has no path to Firestore, so an outage of the messaging service cannot cause it to fail.
Result: PASS
Evidence: `test/nfr/reliability/core-features.spec.ts`

5.6.2 NFR-13: Warm Replica

Requirement: Production maintains at least one warm replica at all times.
Method: Azure CLI inspection of the production container app configuration (`min-replicas` ≥ 1).
Result: PASS

5.7 Maintainability Results

Requirement: Backend test coverage shall be measured on every push and pull request via an automated CI pipeline running against a real, ephemeral PostgreSQL container (not a mock), with results tracked through Codecov. 80% automated coverage is the working target; per the Testing Policy this is not yet an enforced hard gate, and merges are currently gated on the test suite passing and the SonarCloud Quality Gate. Every new endpoint shall have a corresponding entry in the shared `openapi.yaml` the same day it is committed.
Verification method: CI/CD configuration inspection, coverage tooling output, and `docs/OpenAPI/openapi.yaml`.

Clause	Evidence	Result
Coverage measured on every push and PR, with ephemeral Postgres	`.github/workflows/ci.yml` triggers on push and pull_request to `main`. The test job declares a `postgres:16` service container with a healthcheck, and the suite connects to it through `DATABASE_URL`. No mocks or in-memory databases are used.	PASS
Coverage tracked in Codecov	The `codecov/codecov-action` step uploads coverage in CI. Current coverage is 85% (`test/nfr/artifacts/codecov-dashboard.png`).	PASS
Merges gated on tests and SonarCloud Quality Gate	Branch protection on `main` requires the backend test job and the SonarCloud Quality Gate. Pull request #317 shows both checks blocking the merge until green (`test/nfr/artifacts/branch-protection.png`).	PASS
Every new endpoint added to openapi.yaml	A spot check of the most recent pull requests shows `openapi.yaml` updated alongside controller changes (NFR-16).	PASS

Evidence: `test/nfr/artifacts/coverage-summary.txt`, `test/nfr/artifacts/codecov-dashboard` `test/nfr/artifacts/branch-protection.png`.

5.8 Accessibility Results

Requirement: The UI shall meet a Google Lighthouse Accessibility score of at least 85/100 on every audited page, and at least 90/100 on any page introduced from Demo 3 onward.

Method: Google Lighthouse accessibility audits (desktop preset) run against the deployed staging frontend. Each page's raw JSON report was saved as an artifact, and the Lighthouse version and measurement date are recorded in the summary artifact.

Page	URL	Score	85	90 (Demo 3+)	Introduced
Landing	/	94	PASS	PASS	Pre-Demo 3
Auction	/	94	PASS	PASS	Post-Demo 3
Create Listing	/listings/create	94	PASS	N/A	Pre-Demo 3
In-App Messaging	/notifications	95	PASS	N/A	Pre-Demo 3
Wishlist	/wishlist	96	PASS	N/A	Pre-Demo 3
Listings Browse	/listings	89	PASS	N/A	Pre-Demo 3
Audit Log	/admin/audit-log	85	PASS	N/A	Pre-Demo 3

All pages meet the 85 baseline (NFR-17), and the only page introduced from Demo 3 onward (Auction, 94) meets the 90 threshold (NFR-18).

Lowest scorer: Audit Log (85). Its score is driven by dense tabular content, which limits contrast and label opportunities. It is above the baseline and is tracked as a follow-up accessibility improvement. Listings Browse (89) is also tracked for improvement.

Evidence: `test/nfr/artifacts/lighthouse/landing.json` (and an equivalent file for each page) and `test/nfr/artifacts/lighthouse-scores-summary.json`.

5.9 Auditability Results

Requirement: Every moderation, ban and appeal-decision action (approve, reject, ban, uphold, reinstate) shall write exactly one immutable `audit_log` entry recording the action, the performing admin's UUID and a timestamp. This is verified by an integration test for each of UC3, UC8 and UC10 that asserts a matching `audit_log` row exists after the action completes.

Method: Integration tests run against the test database (`textbook_marketplace_test`), exercised through the real HTTP endpoints and the real `audit_log` repository (`test/admin.integration`

Use case	Action	Audit assertions	Test	Result
UC3 Listing moderation	APPROVE_LISTING	Row exists with the action, `entity_id` = listing, `performedBy.id` = admin and a `performed_at` timestamp	Approve a pending listing and create audit log	PASS
UC3 Listing moderation	REJECT_LISTING	Row exists with the action, `entity_id` = listing, `performedBy.id` = admin and the reason populated	Reject a pending listing and create audit log	PASS
UC8 User bans	BAN_USER	Row exists with the action, `entity_type` = USER, `entity_id` = user, `performedBy.id` = admin, the ban reason and a `performed_at` timestamp	Ban a user and create an audit log	PASS
UC10 Appeal decisions	UPHOLD / OVERTURN	Not covered. The appeal decision flow is implemented in `src/cases/`, but no automated test asserts a matching audit row	None	NOT COVERED

Immutability: The `audit_log` entity exposes no update path, and no controller provides mutation endpoints for the table. The `action` column is also constrained by a database CHECK to a fixed set of enumerated values. The tests verify write semantics and cardinality; they do not attempt to mutate existing rows.

Overall verdict: PASS for UC3 and UC8. PARTIAL for UC10, which is tracked as a follow-up item (see Known Issues).
Evidence: `test/admin.integration.spec.ts` and `docs/nfr/auditability-test-run.png` (test run showing the UC3 and UC8 audit tests passing).

5.10 Testing Tools Used

Tool	Purpose	NFRs Covered
k6	Load testing of listing search on staging	NFR-01
Playwright	End-to-end and API tests against staging for notification latency and security clauses	NFR-02, NFR-06, NFR-07, NFR-09, NFR-10, NFR-11
Static code inspection	Verifying configuration and dependency structure in the source	NFR-08, NFR-12
Jest integration tests	Audit log assertions against the real HTTP endpoints and test database	NFR-19, NFR-20
Google Lighthouse	Automated accessibility audits of the staging frontend	NFR-17, NFR-18
Codecov and CI inspection	Coverage tracking, workflow and branch-protection review	NFR-15
Azure CLI	Infrastructure validation for scaling and availability	NFR-04, NFR-05, NFR-13
Architecture and manual review	Stateless design review and OpenAPI specification checks	NFR-03, NFR-16

5.11 Known Issues and Follow-Up Actions

- **NFR-19 / NFR-20 (UC10 appeal decisions):** the appeal decision flow in `src/cases/` has no automated test asserting a matching `audit_log` row.
  - **Action:** add an integration test for the review endpoint that asserts one audit row (action, admin UUID, timestamp) for both the uphold and reverse decisions.
- **Staging rate limit (NFR-01):** the 100 requests/min per-IP limit on staging caps the load that can be generated from a single client, so the load test used 10 virtual users and latency was measured on successful responses only.

- **Accessibility follow-ups (NFR-17):** Audit Log (85) and Listings Browse (89) are above the baseline but are the lowest scorers and are tracked for improvement, mainly contrast and labelling in dense tables.
- **Soft-ban design (NFR-11):** tokens are still issued to banned users so the frontend can render the ban screen. All protected-route denial for banned users therefore depends on the frontend honouring the `is_banned` flag.

6. Deployment

6.1 Overview

The system is deployed as two separate services, a NestJS backend and a Next.js frontend, both packaged into Docker images and run on Azure Container Apps. A managed Azure Database for PostgreSQL Flexible Server handles the data, and Azure Blob Storage handles listing image uploads through the backend, using SAS-token authentication. A Redis container runs inside the same Container Apps Environment and backs the auction job queue, and Azure AI Vision provides text recognition for AI-assisted listing creation.

There are two environments, production (served off `main`) and staging (served off `develop`), which is what covers the environment parity requirement. Both share one Container Apps Environment to keep costs down, but they're fully separated at the application and data level.

	Production	**Staging**
Trigger branch	`main`	`develop`
Backend app	`nexusdev-backend`	`nexusdev-backend-staging`
Frontend app	`nexusdev-frontend`	`nexusdev-frontend-staging`
Database	`nexustextbook`	`nexustextbook_staging`
Scaling	`min-replicas: 1`, always warm	`min-replicas: 0`, scales to zero when idle
Job queue store	`nexusdev-redis` (shared, internal ingress only)	`nexusdev-redis` (shared, internal ingress only)
AI text recognition	`nexusdev-vision` (shared, F0 tier)	`nexusdev-vision` (shared, F0 tier)

Redis and Azure AI Vision are shared by both environments, the same way the Blob Storage account is. Each backend app gets the Redis host and port as environment variables, and the Vision endpoint and key as Container Apps secrets, so the values are set once per app and survive every deploy, because the pipeline only updates the image.

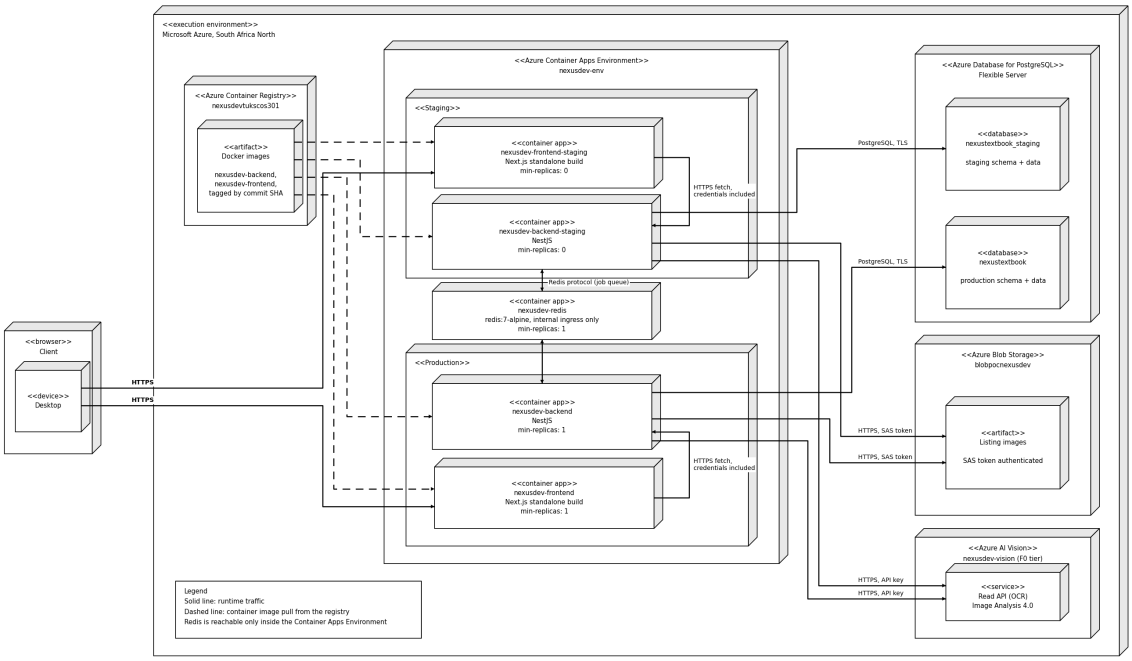
We went with one shared Container Apps Environment and separate databases on purpose, instead of two fully separate environments. A second full environment felt like unnecessary cost for a student capstone project, and the part that actually matters for correctness, keeping the data isolated, is already handled by giving each environment its own Postgres

database on the same server. Staging runs at `min-replicas: 0`, which means the first request after it's been idle a while takes a bit longer (a cold start), but it costs basically nothing while no one's using it. Production stays warm at `min-replicas: 1` since it needs to respond right away for real users at any time.

6.2 Live URLs

- **Production frontend (custom domain):** https://www.unitextbookmarketplace.co.za
- **Production frontend (default hostname):** https://nexusdev-frontend.whitesand-df72b78b.southafricanorth.azurecontainerapps.io
- **Production backend:** https://nexusdev-backend.whitesand-df72b78b.southafricanorth.azurecontainerapps.io/api
- **Staging frontend:** https://nexusdev-frontend-staging.whitesand-df72b78b.southafricanorth.azurecontainerapps.io
- **Staging backend:** https://nexusdev-backend-staging.whitesand-df72b78b.southafricanorth.azurecontainerapps.io/api

6.3 Deployment Diagram

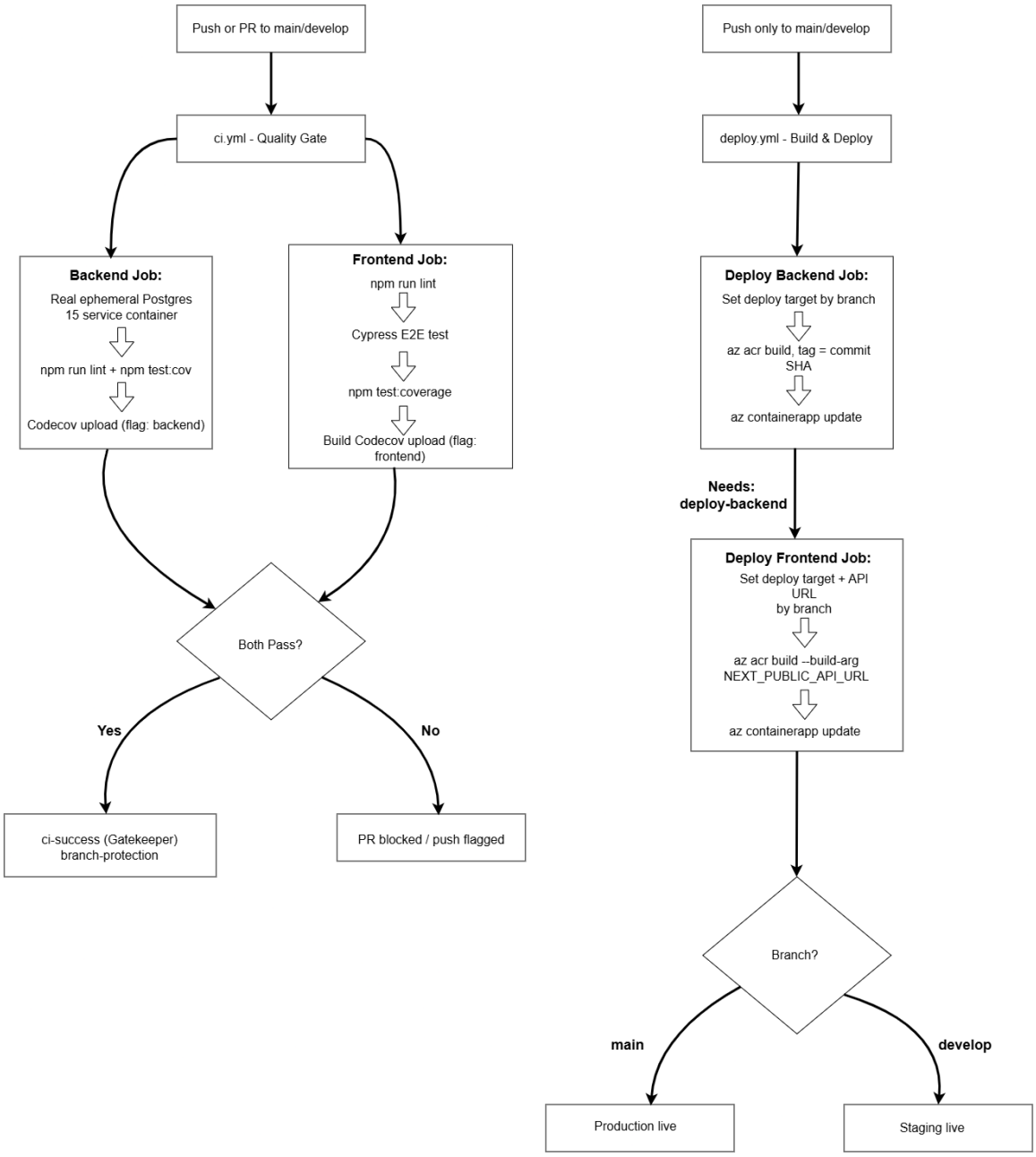


This one diagram covers both environments, since they run on the exact same topology and only differ in which specific app and database instance they point at (labelled per node above). Both backends also talk to the shared `nexusdev-redis` container over the

Container Apps Environment's internal network, and to Azure AI Vision over HTTPS with an API key. Region is South Africa North, resource group is `NexusDev`.

6.4 CI/CD Pipeline Diagram

There are two separate GitHub Actions workflows here, each doing a different job.



A few notes on how this actually works:

- `ci.yml` runs on both push and pull request to `main`/`develop`, this is the real quality gate. It runs the backend and frontend Jest jobs, the Cypress end-to-end specs, and SonarCloud. `deploy.yml` only runs on push, since an open PR shouldn't trigger an actual deployment.

- Branch protection is set up on both `main` and `develop`. Every merge has to come through a PR from a feature branch, needs at least one reviewer to approve it, and the `ci-success` check has to pass. A failed CI run blocks the merge outright.
- Inside `deploy.yml`, `deploy-frontend` only runs after `deploy-backend` succeeds, since the frontend build needs the backend's live URL as a build-time argument.
- Every image is tagged with the full commit SHA, never `:latest`. This was actually a real bug we ran into early on, Container Apps sometimes wouldn't notice that a `:latest` image had changed, so we switched to SHA tags to make every deploy unambiguous.

6.5 Deployment Requirements Status

Requirement	Status	Notes
Live, accessible system	Done	Both environments reachable over public HTTPS (§6.2)
Environment parity	Done	Production and staging each have their own Container Apps, their own database, and their own CI/CD trigger
Infrastructure as Code / Containerization	Done	Docker multi-stage builds, deployed through Azure Container Apps
Secrets management	Done	All credentials live as Container Apps secrets or GitHub Actions secrets, never committed to the repo. This now includes the Vision key and endpoint.
Job queue infrastructure	Done	Redis runs as an internal-only container in the Container Apps Environment, and both backends are configured with its host and port
Custom domain with HTTPS	Done	Frontend served on `www.unitextbookmarketplace.co.za` with a managed certificate (§6.6)
Rollback strategy	Done	See §6.7

6.6 Custom Domain and DNS

The production frontend is also served from https://www.unitextbookmarketplace.co.za. DNS is managed in the registrar's control panel (Absolute Hosting). Two records were added for the `www` host: a TXT record (`asuid.www`) carrying the Container Apps domain

verification ID, and a CNAME record (`www`) pointing at the frontend's default Container Apps hostname. Once both resolved, the hostname was added to the frontend Container App and a free Azure-managed TLS certificate was bound to it using CNAME validation.

Standard DNS does not allow a CNAME on the bare domain, so `www.unitextbookmarketplace.co.za` is the canonical address and is the one used in the README and the demo. Two knock-on changes were needed. The backend's CORS origins in `main.ts` now include the custom domain, and the Blob Storage CORS rules do too, because the photo crop editor reads image pixels through a canvas and would fail on a blocked origin. The backend keeps its default Container Apps hostname, so `NEXT`$_{P}UBLIC_{A}PI_{U}RL, which is baked in at frontend build time, did not need t

6.7 Rollback Strategy

Azure Container Apps keeps old revisions around. If a deployment goes wrong, we can just shift traffic back to the last known good revision without rebuilding anything:

```
az containerapp revision list --name nexusdev-backend --resource-group NexusDev
    --output table
az containerapp ingress traffic set --name nexusdev-backend --resource-group
   NexusDev --revision-weight <previous-revision-name>=100
```

This isn't just theoretical, we actually used this exact process during initial production deployment to fix a stale image that was causing CORS issues.

6.8 Issues Encountered and the Decisions That Came Out of Them

Issue	Root Cause	Decision Made
Stale :latest tag not picked up as changed by Container Apps	The tag string itself never changes, so the platform assumed nothing had	Every image now gets tagged with the commit SHA instead
Cross-origin cookie not keeping people logged in after login	sameSite: 'lax' blocks cross-origin fetch, and frontend and backend end up on different hostnames once deployed	Cookie settings now change per environment, sameSite: 'none' and secure: true when deployed, 'lax' locally

Issue	Root Cause	Decision Made
Fresh database had no seed data, so registration kept failing	The seed scripts had never actually been run against a genuinely empty database before	Confirmed the seed scripts work properly against both a fresh staging database and a patched production database
Two migrations failed the same way on staging and, it turned out, on production too	Migration class names didn't match their required timestamp suffix, and a leftover name property on the class kept overriding the fix even after the class name was already corrected	Fixed both migration files properly, and manually applied the corrected SQL straight to production since it had the exact same problem
The frontend image accidentally got deployed into the backend's Container App on staging	A copy-paste mistake in deploy.yml's branch logic set the wrong app name in one spot	Fixed the value, then double-checked with az containerapp show that each app was finally running the right image
Staging was costing money even while nobody was using it	min-replicas: 1 keeps a Container App running the whole time no matter what	Set min-replicas: 0 on both staging apps, accepting a short cold start on the first request after it's been idle
Auctions needed to close at an exact time, and anti-sniping keeps moving that time	A polling cron job wastes work, and cancelling and re-adding a delayed job races when two extensions land close together	Event-driven BullMQ jobs, with a close job that re-checks the end time and reschedules itself, so it can never fire early or twice

Issue	Root Cause	Decision Made
Bid transaction failed with "FOR UPDATE cannot be applied to the nullable side of an outer join"	The locking query joined the seller and highest-bidder relations, and Postgres refuses to lock a nullable joined side	Removed the joins from the locked query and exposed the foreign key ids as plain columns on the entities
Auction feature could not run on Azure because there was no Redis there	Redis existed only as a local Docker container	Provisioned Redis as an internal-only container app in the existing environment. We chose it over a managed Redis service because that needs TLS on a different port, a code branch between dev and production, and a monthly cost
Background removal model stalled and was far too heavy for phones	The ONNX model was roughly 176MB and timed out after 30 seconds	Dropped it for a dependency-free flood fill that runs in the browser and falls back to manual corners on busy backgrounds
Crop confirm could fail with a tainted canvas	Photos load from Blob Storage and are read through a canvas, which needs CORS headers	Added every deployed frontend origin, including the custom domain, to the storage account's CORS rules

Uni Textbook Marketplace

In collaboration **with:**





2026 NexusDev - University of Pretoria - COS 301 Capstone Project
This document is version-controlled in the GitHub repository under
/docs/Demo_4/Software Architecture Specifications.pdf